

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1105

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05

Total Marks:25

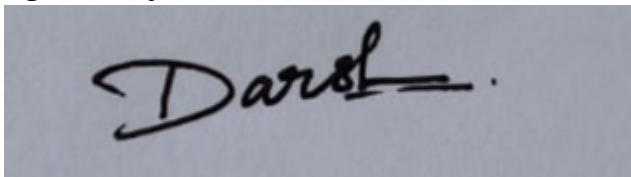
Annexure A

Error Analysis Task

Code of Conduct:

I DARSHINIE KARANL.V.M (192520033) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A photograph of a handwritten signature in black ink on a light blue background. The signature appears to be 'Darshinie' followed by a horizontal line and a period.

RUBRICS FOR CALCULATION:

Error Analysis Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Annexure A

Error Analysis Task

Q1. Scenario: Digital Wallet System

Modules:

User Registration, Wallet Authentication, Money Transfer, Transaction History

Question:

Develop **pseudocode for an error analysis and quality assurance process** that validates secure wallet authentication and money-transfer operations. Include input validation, authentication failure handling, transaction consistency checks, exception handling, error classification, and test-result logging.

Q2. Scenario: Online Food Delivery Platform

Features:

Restaurant Search, Menu Selection, Shopping Cart, Order Placement, Online Payment

Question:

Design **pseudocode for detecting and analyzing usability errors** in the food delivery platform. The process should measure restaurant search time, menu navigation errors, cart modification errors, checkout completion time, and payment-related failures. Explain how the algorithm identifies usability problems and generates an evaluation report.

Q3. Scenario: University Course Registration System

Modules:

Student Authentication, Course Search, Course Registration, Timetable Management

Question:

Write **pseudocode to generate and execute test cases** for course registration. Include validation of student eligibility, course prerequisites, seat availability, timetable conflicts, duplicate registration, and invalid input. Show how detected errors are recorded and reported.

Q4. Scenario: Smart Parking Management System

Modules:

User Registration, Parking Slot Search, Slot Reservation, Payment, Vehicle Entry/Exit

Question:

Develop **pseudocode for constraint-based error analysis** in the parking reservation process. The booking must satisfy conditions such as valid vehicle details, available parking slots, valid reservation duration, and successful payment. Demonstrate how invalid conditions are detected, handled, and logged.

Q5. Scenario: Online Examination System

Features:

Student Login, Exam Selection, Question Display, Answer Submission, Result Processing

Question:

Write **pseudocode for integrating quality assurance and usability testing** in the online examination system. The process should verify functional correctness of login, question delivery, answer submission, and result processing while also evaluating response time, navigation difficulty, accessibility, and user errors. Explain how the collected results can be analyzed to identify system defects and usability issues.

ANSWERS

1.

The Digital Wallet System consists of User Registration, Wallet Authentication, Money Transfer, and Transaction History. The purpose of the quality assurance process is to identify errors, verify secure authentication, ensure correct money transfers, maintain transaction consistency, and record all test results systematically. The process should particularly focus on input validation, authentication failures, transaction consistency, exception handling, error classification, and test-result logging.

Input Validation

Input validation is the first step in the QA process. It ensures that only valid information is processed by the wallet system.

The system should validate:

- User ID and wallet ID.
- Authentication credentials such as password, OTP or biometric information.
- Sender and receiver account details.
- Transfer amount.
- Transaction-related information.

The system should reject:

- Empty or missing fields.
- Invalid account numbers.
- Negative or zero transaction amounts.
- Invalid user credentials.
- Transfer requests where the sender and receiver are the same.
- Transactions exceeding the available wallet balance.

Every invalid input should generate an appropriate error message and should be recorded in the test log.

Secure Wallet Authentication

Authentication is important because financial transactions should only be performed by authorized users.

The QA process should test:

- Successful login with valid credentials.
- Login with an incorrect password.
- Invalid or expired OTP.
- Multiple incorrect authentication attempts.
- Biometric authentication failure.
- Unauthorized access to another user's wallet.
- Session timeout after inactivity.

If authentication fails, the system should deny access to the wallet or transaction function.

After repeated failed attempts, the system may temporarily lock the account to prevent unauthorized access. The authentication failure should also be recorded for further security analysis.

Authentication Failure Handling

Authentication failures should be handled safely without exposing sensitive information.

For example:

User enters incorrect OTP → System rejects authentication → Error message displayed → Failed attempt recorded

The system should:

1. Reject the invalid authentication attempt.
2. Display a clear but non-sensitive error message.
3. Increase the failed-attempt counter.
4. Allow another attempt if the limit has not been reached.
5. Temporarily lock the account after repeated failures.
6. Record the failure in the security/test log.

This prevents unauthorized users from accessing wallet information and performing transactions.

Money Transfer Validation

After successful authentication, the system validates the money-transfer request.

The following conditions should be checked:

- Sender account exists.
- Receiver account exists.
- Sender is authorized to make the transaction.
- Transfer amount is valid.
- Sender has sufficient balance.
- Transaction request is not duplicated.

For a valid transaction:

Sender Wallet → Debit Amount → Receiver Wallet → Credit Amount → Update Transaction History

The transaction should be considered successful only when all required operations are completed.

Transaction Consistency Checks

Transaction consistency is essential because incorrect balance information can cause financial loss.

For example, if a user transfers ₹2,000:

- Sender's balance must decrease by ₹2,000.
- Receiver's balance must increase by ₹2,000.
- Transaction history must show ₹2,000.
- Sender and receiver details must be correctly stored.
- The transaction status should be marked correctly.

The system should compare the balances before and after the transaction.

If any inconsistency is detected, the transaction should not be completed.

Example

Before Transaction

Sender = ₹10,000

Receiver = ₹5,000

Transfer = ₹2,000

After Transaction

Sender = ₹8,000

Receiver = ₹7,000

If the sender is debited but the receiver is not credited, the transaction is inconsistent. The system should rollback the transaction and restore the original balances.

Transaction Rollback

Rollback is used when an error occurs during a money transfer.

For example:

Debit Sender → Database Failure → Credit Receiver fails

In this situation, the system must not leave the sender with ₹2,000 deducted without crediting the receiver.

Therefore:

Error occurs → Rollback → Restore previous balance → Record failure → Notify user

This maintains data consistency and prevents financial discrepancies.

Exception Handling

The system should use exception handling to manage unexpected failures.

Possible exceptions

Database Exception

- Database becomes unavailable during a transaction.
- Solution: Rollback the transaction and record the error.

Network Exception

- Connection is lost during money transfer.
- Solution: Check transaction status before retrying to prevent duplicate transactions.

Authentication Exception

- Authentication service fails.
- Solution: Stop the transaction and display an appropriate error.

Transaction Exception

- Debit or credit operation fails.
- Solution: Rollback all related operations.

Unexpected Exception

- Any unknown system failure.
- Solution: Safely terminate the operation, log the error and prevent incorrect wallet updates.

Error Classification

After detecting an error, it should be classified according to its type.

Input Error

Example: Invalid wallet ID or negative amount.

Authentication Error

Example: Incorrect password or OTP.

Security Error

Example: Unauthorized user attempting to access another wallet.

Data Integrity Error

Example: Sender's balance is deducted but receiver's balance is not updated.

Transaction Error

Example: Money transfer fails during processing.

Concurrency Error

Example: Two users attempt to transfer money from the same wallet simultaneously.

System Error

Example: Database or server failure.

Error classification helps developers determine the cause, severity and priority of defects.

Concurrency and Duplicate Transaction Testing

Multiple users may access the wallet system at the same time. Therefore, the QA process should test simultaneous transactions.

Important test cases include:

- Two transfers from the same wallet at the same time.
- Two users attempting to transfer the same available balance.
- Multiple requests submitted for the same transaction.
- Repeated clicking of the transfer button.
- Network retry after a delayed response.

The system should prevent:

- Double spending.
- Duplicate transactions.
- Incorrect wallet balances.
- Lost transaction records.

Proper transaction control and synchronization should be used to maintain consistency.

Transaction History Verification

Every successful transaction should be recorded correctly.

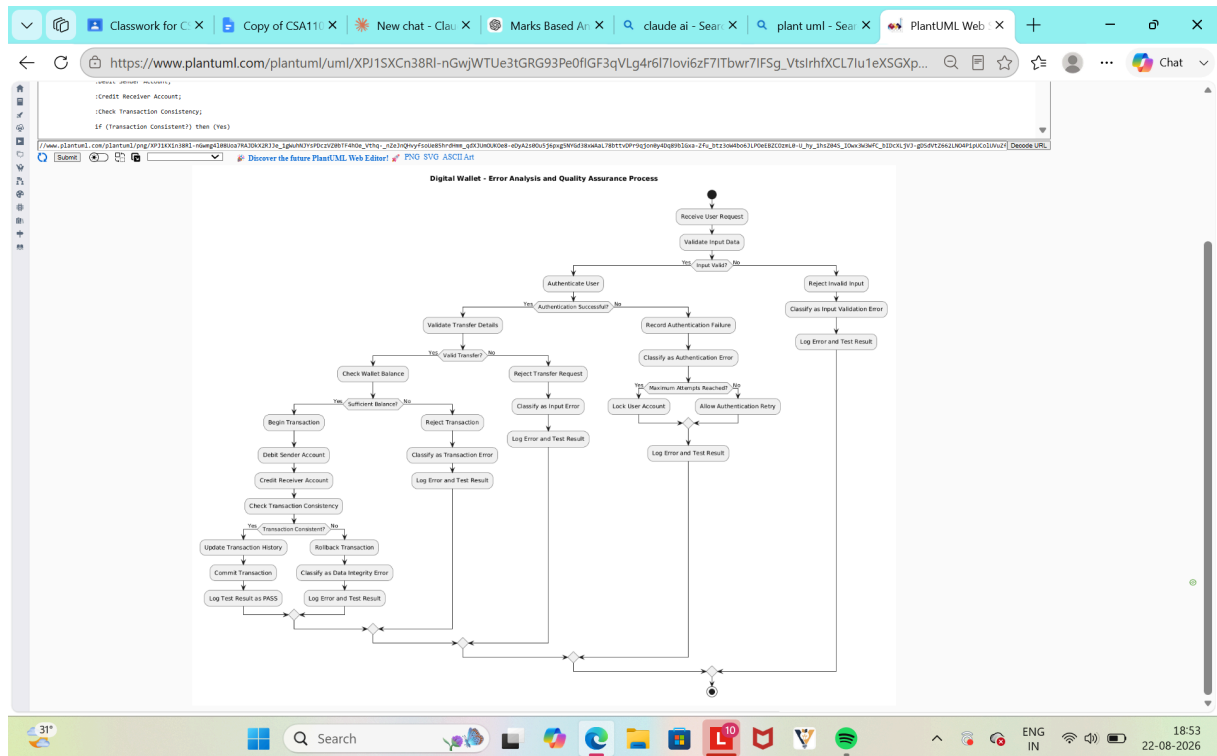
The transaction history should contain:

- Transaction ID.
- Sender.
- Receiver.
- Amount.
- Date and time.
- Transaction status.

QA testing should verify that:

- Successful transactions appear in history.
- Failed transactions are not incorrectly shown as successful.
- Duplicate records are not created.
- Transaction details match the actual wallet balances.

Activity diagram



Error Analysis Process

The overall error analysis process can be represented as:

Input Received



Validate Input



Authenticate User



Authentication Successful?

→ No: Record Authentication Error → Reject Request

→ Yes: Continue



Validate Transfer Details



Check Wallet Balance



Begin Transaction



Debit Sender



Credit Receiver



Check Transaction Consistency



Consistency Valid?

→ No: Rollback → Classify Error → Log Result
→ Yes: Commit Transaction
↓
Update Transaction History
↓
Log Test Result
↓
Transaction Completed

Quality Assurance Benefits

This QA process provides the following benefits:

- Prevents unauthorized wallet access.
- Detects invalid user inputs.
- Prevents incorrect money transfers.
- Maintains accurate wallet balances.
- Prevents duplicate transactions.
- Handles system and database failures safely.
- Maintains accurate transaction history.
- Helps identify and classify defects.
- Provides proper records of testing activities.
- Improves the overall security, reliability and quality of the wallet system.

Conclusion

The Digital Wallet System requires a systematic quality assurance and error analysis process. Input validation prevents incorrect data, while secure authentication prevents unauthorized access. Transaction consistency checks and rollback mechanisms ensure that money transfers remain accurate. Exception handling protects the system from unexpected failures, while error classification and test-result logging help identify, track and correct defects. Therefore, the proposed process improves the security, accuracy, reliability and overall quality of the Digital Wallet System.

2.

The Smart Parking Management System consists of User Registration, Parking Slot Search, Slot Reservation, Payment, and Vehicle Entry/Exit. The reservation process should satisfy important constraints such as valid vehicle details, availability of the selected parking slot, valid reservation duration, and successful payment. The purpose of constraint-based error analysis is to identify invalid conditions before completing a reservation, handle the errors appropriately, and maintain a record of all errors and reservation results.

Vehicle Details Validation

The first step is to verify the vehicle information provided by the user.

The system should check:

- Vehicle number is entered.

- Vehicle number follows the required format.
- Vehicle type is selected.
- Vehicle details are associated with the registered user.

If the vehicle number is missing or invalid, the system should:

1. Reject the reservation request.
2. Display an appropriate error message.
3. Classify it as a Vehicle Validation Error.
4. Record the error in the error log.

Example:

If the user enters an invalid vehicle number, the system displays "Invalid vehicle details. Please enter a valid vehicle number."

Parking Slot Availability Check

After validating the vehicle, the system checks whether the selected parking slot is available.

The system should verify:

- Selected slot exists.
- Slot is currently available.
- Slot is not already reserved.
- Slot is suitable for the selected vehicle type, if applicable.

If the selected slot is unavailable:

- The reservation should not proceed.
- The user should be informed that the slot is unavailable.
- The error should be classified as a Slot Availability Error.
- The failure should be recorded in the log.
- The system can display alternative available slots.

Example:

User selects Slot A12 → Slot A12 already reserved → Booking rejected → Error logged

Reservation Duration Validation

The reservation duration must be valid before calculating the parking fee.

The system should check:

- Start time is provided.
- End time is provided.
- End time is later than start time.
- Duration is greater than zero.
- Duration does not exceed the permitted parking period, if such a limit is defined.

Invalid conditions

- Start time is missing.
- End time is missing.
- End time is earlier than start time.
- Start and end time are the same.
- Duration is negative or zero.

If an invalid duration is detected, the system should reject the reservation and record a Duration Validation Error.

Payment Validation

After the vehicle, slot and duration have been successfully validated, the system calculates the parking fee and processes the payment.

The payment process should verify:

- Correct payment amount.
- Valid payment method.
- Successful payment response.
- Payment transaction status.

If payment succeeds:

Payment Successful → Reservation Confirmed → Slot Marked as Reserved

If payment fails:

Payment Failed → Reservation Cancelled → Slot Remains Available → Error Logged

The system should never confirm a parking reservation when the payment has failed.

Error Handling

When an error occurs, the system should handle it in a controlled manner.

The general process is:

Detect Error → Classify Error → Display Message → Stop/Correct Operation → Log Error

For example:

Invalid Vehicle Details

- Display error message
- Reject reservation
- Record error

Unavailable Slot

- Display unavailable status
- Suggest another slot
- Record error

Invalid Duration

- Request corrected time
- Do not proceed to payment
- Record error

Payment Failure

- Cancel reservation
- Keep slot available
- Record payment error

Error Logging

Every detected error should be stored in an error log for future analysis.

The log may contain:

- Error ID
- User ID
- Vehicle number
- Parking slot
- Error type
- Error description
- Date and time
- Reservation status

Example Error Log

Error ID	Error Type	Description	Status
E01	Vehicle Error	Invalid vehicle number	Failed
E02	Slot Error	Slot already reserved	Failed
E03	Duration Error	Invalid reservation duration	Failed
E04	Payment Error	Payment unsuccessful	Failed
E05	System Error	Reservation service unavailable	Failed

Logging helps administrators identify repeated problems and improve the parking system.

Successful Reservation

A reservation should be confirmed only when all constraints are satisfied.

The successful process is:

Valid User

↓

Valid Vehicle Details

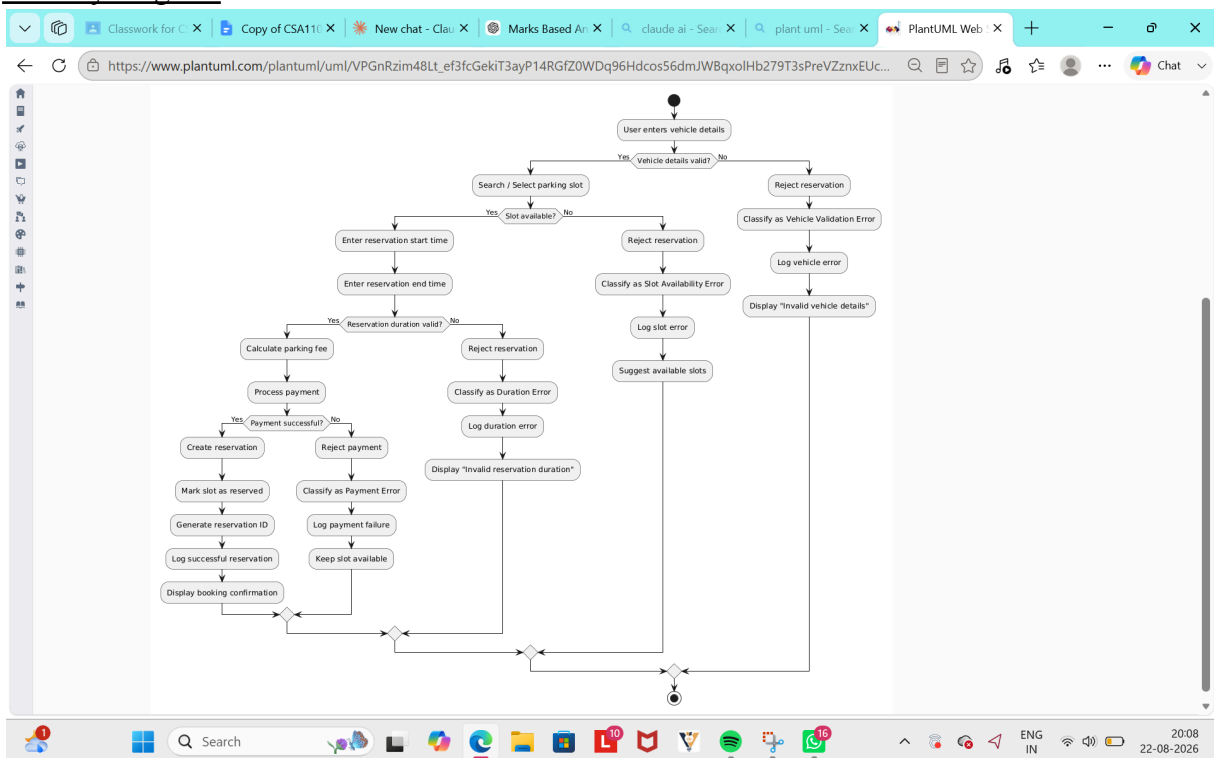
↓

Available Parking Slot
 ↓
 Valid Reservation Duration
 ↓
 Successful Payment
 ↓
 Create Reservation
 ↓
 Mark Slot as Reserved
 ↓
 Generate Confirmation

The system should provide the user with:

- Reservation ID
- Parking slot number
- Vehicle details
- Start time
- End time
- Payment status

Activity diagram



Benefits of Constraint-Based Error Analysis

This approach provides several benefits:

- Prevents invalid reservations.
- Avoids booking occupied parking slots.
- Prevents incorrect reservation durations.

- Ensures payment is completed before confirmation.
- Reduces user errors.
- Maintains accurate reservation records.
- Helps identify recurring system defects.
- Improves reliability and consistency.
- Provides clear feedback to users.

Conclusion

Constraint-based error analysis ensures that a parking reservation is completed only when all required conditions are satisfied. The system validates vehicle details, checks slot availability, verifies reservation duration and confirms successful payment. When an invalid condition occurs, the reservation is stopped, an appropriate error message is provided, the error is classified, and the event is logged. This process improves the accuracy, reliability, security and user experience of the Smart Parking Management System.